



PRODUCT & TECHNICAL ROADMAP

From a till to a **restaurant operating system**

The phased plan for evolving Flames POS — order pipeline, staff & cash discipline, kitchen operations, inventory through procurement, and guest ordering — benchmarked against Blink POS and grounded in a 57-finding audit of the current build.

Where Flames POS stands, and where it goes

Flames POS today is a competent single-branch till: point of sale with running tabs and rounds, a genuinely strong kitchen display, order history, basic analytics, menu management, thermal receipt printing and a public menu. Blink POS — the benchmark for this plan — is a restaurant **operating system**: the same core plus shift & cash discipline, per-person staff accounts, recipe-level inventory, procurement, sales channels and expense tracking, organised into nine module groups.

4/9

BLINK MODULE GROUPS
COVERED IN PART TODAY

4

MISSING ARCHITECTURAL
FOUNDATIONS

57

AUDIT FINDINGS FEEDING
THIS PLAN

7

DELIVERY PHASES, P0-P6

The gap is not user-interface polish. It is four missing foundations, and almost every capability stakeholders want arrives blocked by at least one of them:

- 01 **Order line items live inside a JSON blob.** Per-dish analytics, automatic stock deduction, kitchen-station routing and item-level bill splitting all need real line records with identity.
- 02 **There is no payments ledger.** One "payment mode" field per order rules out split bills, mixed tender, drawer reconciliation and channel settlement.
- 03 **Two shared PIN accounts.** Without per-person identity there is no attribution, no approvals, no shift ownership and no honest audit trail.
- 04 **No branch dimension.** Every table built without it becomes retrofit work the day a second branch opens.

The strategy in one sentence: stabilise the money path first (twelve quick fixes), lay the four foundations second (one schema wave, one transactional layer), then build features in dependency order — because rushing features onto the current schema is the one path that guarantees paying for the same work twice.

Scope decisions already taken with ownership: QR dine-in ordering is *in* the roadmap; inventory goes the full distance to procurement (suppliers, purchase orders, goods-received); aggregator orders get channel tracking now with the Foodpanda API integration deferred until partner access exists.

What Flames POS is today

Next.js 16 + Supabase (Postgres, row-level security, realtime, storage) on Vercel; roughly 8,000 lines. A recent 57-finding audit — ten independent review passes, each finding adversarially challenged — grades the build.

MODULE	STATE
POS till	STRONG Cart, variants & modifiers, dine-in / takeaway / delivery, waiter & table, customer capture by phone, discounts, tabs with rounds, duplicate-safe checkout (partial), silent thermal printing
Kitchen display	STRONG Three lanes, first-in-first-out, round badges, per-ticket timers with 5/10-minute escalation, new-order chime, realtime + polling
Orders	STRONG Server-side paging, filters, search, sort; reprint; void with mandatory reason (unpaid only)
Reports	BASIC Revenue / orders / average order value with trend, payment mix, busiest hours, top items, revenue per waiter
Menu	GOOD Item & category management, image upload, one-tap 86. Modifier definitions have no management screen
Settings	THIN Seven fields in total (merchant identity, payment QR, tax rate & label, auto-print)
Public menu	GOOD Read-only, freshly re-photographed (85 dishes), grid & list views
Auth	LIMITING Two shared accounts (admin / staff), six-digit PIN, role-gated routes

Debt that must precede feature work

Highest-impact audit findings: a settle path that can bill unsent items; round-sending that can double-fire on retry; the duplicate-protection key lost on natural retry paths; a tab's agreed discount silently dropped at settle; unpaid tabs counted as collected revenue; day boundaries computed in UTC (the restaurant is UTC+5); invoice numbers that can collide and are sometimes never stored; database permissions that let any signed-in till edit any order (the admin-only gate is client-side); one terminal's logout signing out every terminal; and a kitchen board that re-downloads the entire order history every fifteen seconds.

Worth protecting: the tabs-and-rounds model (Blink's "running orders", already done well), the KDS experience, the hard-won receipt pipeline, server-side order pagination, the black brand theme, and the security-first posture on menu data.

What Blink POS teaches

A logged-in walkthrough of a live Blink deployment (single branch, admin access) mapped nine sidebar groups: Order Management, Menu, Inventory, Warehouse, Analytics & Reports, Sales Channels, Expenses, Settings, Support. The technology is older than ours — server-rendered Laravel — so the lesson is not the stack; it is the **operational model**.

Blink's order lifecycle, A to Z

menu defined → shift opened with float → order created (any channel) → KOT fires per station rules → payment settles with charges / discounts / vouchers → **stock auto-depletes via recipes** → low stock triggers purchase order → everything rolls into one reporting trail

Flames today implements steps 1, 3, 4 (screen only) and 5 (partially) — and nothing of shift opening, stock depletion or replenishment. That single sentence is most of this roadmap.

Core vs advanced

Core, every restaurant: the till, order list & reversal, dine-in tables and waiters, KDS with station screens, shift schedule, menu with variations and deals, charges / discounts / vouchers / credit, and a dashboard that shows net sales, cancelled orders, expected cash and active shifts.

Advanced, scale features: recipe inventory and stock counts, the warehouse layer (suppliers, purchase orders, transfers, supplier balances), marketplace and aggregator channels, rider fleet, food-cost and BI reporting, multi-branch controls.

Judged not relevant for Flames now

Blink Marketplace (their own aggregation product), Blink-branded loyalty, branch-wise statistics and per-branch menu toggles (until a second branch is real), "Air Inventory" (semantics unclear even in their UI), the help-desk module, multi-language admin.

Master Settings — a lesson in itself

One switchboard governs KOT rules, customer-prompting, per-item instructions, drawer rules, tax before/after discount, card-reference requirements, sounds and waste-reason enforcement. Flames' equivalent is seven fields; this roadmap grows it deliberately, one section per phase.

Capability by capability

HAVE solid today **IMPROVE** exists, needs work **ADD** missing, recommended **NOT NOW** deliberately out of scope.

AREA	CAPABILITY	STATUS	NOTES
Ordering	Till / cart / order types	IMPROVE	Solid; fix audit defects; add per-item notes and price guards
Ordering	Tabs, running orders, rounds	IMPROVE	Good model; make round-append and settle atomic (P1)
Ordering	Order history & filters	HAVE	Add channel filter once channels land
Ordering	Void / reversal	IMPROVE	Unpaid-only today; add manager approval and paid-refund flow
Ordering	Split bill / multi-tender	ADD	Needs the payments ledger (P1)
Ordering	Order channel tracking	ADD	Walk-in / Foodpanda / phone at the till; API later
Floor	Table map, statuses, transfer	ADD	Table number is free text today
Floor	Waiter assignment	IMPROVE	Exists; folds into the staff model (P2)
Kitchen	KDS lanes / timers / rounds	HAVE	Fix full-table polling (P0)
Kitchen	Station screens & KOT routing	ADD	Per-station tickets, printed or on screen (P4)
Kitchen	Prep-time metrics	ADD	Data already captured, never aggregated
Cash	Shifts, float, X/Z, over-short	ADD	The single biggest gap vs Blink (P2)
Cash	Cash movements & expenses	ADD	Paid-in/out, drops, petty cash (P2)
Staff	Per-person accounts, PIN switch	ADD	Foundation; replaces shared PINs (P2)
Staff	Granular permissions	ADD	Void / discount / refund / cash rights (P2)
Staff	Audit log	ADD	Append-only trail on money, menu, settings (P1)
Menu	Items, categories, variants	HAVE	Add sort order, cost price, station tag
Menu	Modifier management UI	ADD	Table exists; screen missing (P3)
Menu	Deals / combos	ADD	Bundle pricing that keeps KDS and stock correct (P3)
Pricing	Discount presets, approval, limits	IMPROVE	Ad-hoc discount exists; policy does not (P3)
Pricing	Vouchers / promo codes	ADD	P3
Pricing	Service charge / delivery fee	ADD	Configurable charge lines (P3)
Pricing	Credit customers (khata)	ADD	Common for corporate clients in Pakistan (P3)
Inventory	Ingredients, recipes, deduction	ADD	Owner chose full depth (P5)
Inventory	Counts, variance, waste, auto-86	ADD	P5
Procurement	Suppliers, POs, GRN, balances	ADD	Owner chose to include (P5)
Reporting	Dashboard KPIs	IMPROVE	Fix revenue & timezone defects; add expected cash, channel and category splits
Reporting	Food-cost dashboard	ADD	After inventory (P5)

AREA	CAPABILITY	STATUS	NOTES
Reporting	Day-end (Z) report	ADD	With shifts (P2)
Customers	Profiles, history, search	IMPROVE	Data captured today; no screen (P3)
Guests	QR dine-in ordering	ADD	Owner-selected; reuses menu + KDS (P6)
Guests	Full online ordering & payment	NOT NOW	Long-term vision; blocked on merchant account
Channels	Foodpanda API integration	NOT NOW	Later; tracking field first
Delivery	Rider assignment (light)	ADD	Rider on delivery orders + status (P3)
Platform	Multi-branch readiness	ADD	Schema readiness only – two rules (P1)
Platform	Notifications	IMPROVE	KDS chime exists; add order-ready, low-stock, shift alerts
Platform	Devices / printer registry	ADD	With KOT routing (P4)

The ten moves that matter most

- 01 **Atomic order pipeline with real line items** (P1) — unlocks everything below; retires the double-billing defect class.
- 02 **Payments ledger** (P1) — split bills, mixed tender, drawer maths and channel settlement all hang off it.
- 03 **Per-person staff with two-second PIN switching and permissions** (P2) — attribution, approvals, an honest audit trail.
- 04 **Shift & cash management** (P2) — float, X/Z reports, over-short. The expected-cash discipline pays for itself in month one.
- 05 **Inventory: recipes → auto-deduction → waste → food cost** (P5) — the menu becomes a costed product.
- 06 **Procurement: suppliers, purchase orders, goods-received** (P5) — closes the loop from stock to purchasing.
- 07 **KOT printing and station routing** (P4) — a paper trail per station; the kitchen becomes stations, not one board.
- 08 **Charges, discount presets with approval, vouchers** (P3) — money rules become configured policy, not till habit.
- 09 **Channel tracking and a customers screen** (P3) — aggregator orders reported honestly; captured customer data made visible.
- 10 **QR dine-in ordering** (P6) — guests order from the table into the existing kitchen flow; payment stays at the till.

Seven phases, dependency-ordered

Sizes: **S** ≈ a day or less · **M** ≈ two to five days · **L** ≈ one to three weeks. Every phase ships to production and runs in live service before the next begins.

P0 — Stabilise now

No schema changes; ship within a week. The audit fixes that do not overlap later foundation work.

Guard the settle path

CRITICAL S

Block settling a tab while unsent items sit in the cart — the overcharge scenario.

pos till · one guard

Six-digit PIN enforcement

HIGH S

A malformed PIN can currently lock a role out of the till.

profile

Fix kitchen-board polling

CRITICAL S

Fetch only active tickets instead of the entire order history every 15 seconds.

KDS query + index

Per-terminal logout

HIGH S

One terminal signing out no longer signs out the till and kitchen.

auth

Keep the duplicate-protection key

HIGH S

Persist the checkout idempotency key with the cart so retries never double-ring.

pos till

Truthful reprints

HIGH S

Original order date + REPRINT watermark.

receipt

Restore tab discounts at settle

HIGH S

An agreed discount must survive to the final bill.

pos till

Fix blank report export

HIGH S

Print rules currently blank every non-receipt page.

print CSS

Honest revenue numbers

HIGH S

Unpaid tabs leave the revenue KPIs; own tile instead.

reports

Reset discounts between orders

MEDIUM S

A comp must never carry to the next customer.

pos till

Pakistan-time reporting

HIGH S

Day and hour buckets in Asia/Karachi, not UTC.

reports (interim)

Contrast & touch targets

MEDIUM S

Readable text on brand orange; 44px cart controls.

theme

P1 — Transactional core

The keystone. One migration wave, one transactional layer; the screens barely change, and everything later stops being blocked.

Order line items

CRITICAL L

Real per-line records (with round grouping) become the source of truth; the old JSON blob is maintained as a receipt snapshot during transition, so every screen keeps working while readers migrate.

new: order_rounds, order_items · backfilled from history

Payments ledger

CRITICAL M

Every tender is a row — method, amount, who took it, which shift. History backfilled; partial payment becomes possible.

new: payments · orders gain “partial” status

Atomic order operations

CRITICAL L

Create, add-round, settle and void become single database transactions: locked, idempotent, totals recomputed server-side and checked against what the cashier saw. Two audit criticals die here by design.

Postgres functions · client write access to money tables revoked

Real invoice numbers

HIGH S

Sequential per day, unique, always stored.

new: invoice_counters

Audit log

HIGH M

Append-only trail written inside the same transactions: voids, discounts, settles, price changes.

new: audit_log + admin viewer

Branch readiness

HIGH S

A one-row branches table; every new table carries branch_id from day one. Two rules, no UI — the retrofit trap avoided.

new: branches

P2 — People & cash custody

The first visibly Blink-class capability: named cashiers and a disciplined drawer.

Staff accounts + PIN switching

CRITICAL M

Terminals stay signed in; people switch in two seconds with a personal PIN, verified server-side.

new: roles, staff, staff_sessions · till lock screen

Permission enforcement

HIGH M

Void, discount, refund and cash rights per role; manager-PIN override at the cashier's till.

checks inside P1 functions

Shifts & drawer

CRITICAL L

Open with a float; X report mid-shift; Z at close with counted vs expected cash and over-short, frozen forever.

new: shifts, cash_movements · shift screens

Expenses (light)

MEDIUM S

Petty cash from the drawer appears in both drawer maths and the P&L.

new: expenses

Waiters become staff

MEDIUM S

One people table; order attribution intact.
migration

Staff management screen

HIGH S

Add, deactivate, assign roles, reset PINs.
admin UI

P3 — Selling surface

Money rules, channels and customer visibility.

Reports on SQL

HIGH M

Aggregation moves into the database; permanent timezone fix; category & channel splits, expected-cash tile.

views over line items + payments

Split bills

MEDIUM M

By amount free with the ledger; by item via allocations.

new: payment_allocations

Charges

HIGH M

Service charge / delivery fee as configured lines.

new: charges, order_charges

Vouchers

MEDIUM M

Code-based redemption with expiry and usage limits.

new: vouchers

Discount presets & policy

HIGH M

Named presets, maximum %, manager approval on override.

new: discount_presets

Deals / combos

MEDIUM L

Bundle price exploding into component lines so kitchen and stock stay correct.

new: deals, deal_items

Channels

HIGH S

Walk-in / Foodpanda / phone keyed at the till; honest per-channel reporting.

new: channels · orders.channel_id

Credit customers (khata)

MEDIUM M

Credit tender plus balance sheet per account.

new: credit_accounts

Customers screen

HIGH M

List, search, order history, export — the data is already captured.

new route

Riders (light)

MEDIUM S

Assign a rider to delivery orders; dispatched / delivered states.

staff role + order fields

Per-item notes

MEDIUM S

“No onions” per line, at the till and in the kitchen.

order_items.notes

P4 — Floor & kitchen

The room and the line, modelled properly.

Table map

HIGH M

Real tables with states — free, seated, billed — plus transfer and merge.

new: floor_tables

KDS on live lines

MEDIUM M

The board subscribes to rounds and lines; per-line bump becomes possible.

uses P1 tables

Stations

HIGH M

Grill, tandoor, fry, dessert: items route to their station; the KDS filters per screen.

new: stations · item station tags

Prep-time metrics

MEDIUM S

Fired-to-bumped times, already captured, finally aggregated.

small dashboard

KOT printing

HIGH L

A paper ticket per station per round, with reprint logging and a printer registry.

new: kot_prints, devices · hardware required

P5 — Inventory → procurement

Owner-selected full depth: from recipe to purchase order.

Ingredients & recipes

HIGH M

Ingredient master with units and cost; per-dish recipes including sub-recipes.

new: ingredients, recipes

Low stock & auto-86

MEDIUM S

An ingredient running out 86s its dishes and prompts reorder.

thresholds + alerts

Stock ledger & auto-deduction

HIGH M

Every sale deducts through the recipe; stock is a ledger of movements, not an editable number.

new: stock_moves · hooks in order functions

Food-cost dashboard

HIGH M

Theoretical vs actual cost, waste cost, margin per dish.

views

Counts, variance & waste

HIGH M

Periodic counts against theoretical stock; waste logged with reasons.

count screens · waste capture

Suppliers, POs & GRN

HIGH L

Suppliers, purchase orders, goods-received into stock, supplier balances.

new: suppliers, purchase_orders, grn

P6 — Growth

Guests, alerts and the door to scale.

QR dine-in ordering

HIGH L

A table QR opens the public menu with a cart; the order lands unpaid on the kitchen board tied to the table; payment stays at the till — no gateway required.

guest sessions · reuses order pipeline

Notifications

MEDIUM M

Order-ready, low-stock, shift-not-closed, day-end summary.

settings-driven

Foodpanda API

MEDIUM L

Orders landing automatically as pending; menu sync. Requires partner access.

webhooks · external dependency

Loyalty (lite)

LOW M

Visit counts and simple rewards on the customer ledger.

customer screen

Branch #2 activation

LOW M

At this point: insert a branch row, per-branch settings, a branch switcher. The schema fights none of it.

P1 readiness pays off

Eight decisions, made once

- 01 **Line-item rows become canonical; the JSON blob demotes to a receipt snapshot.** Rows carry identity (a line can be bumped, voided, allocated, deducted); the snapshot keeps every existing screen working while readers migrate — reports first, kitchen second, receipts last.
- 02 **A payments table, not wider columns.** Methods cash / card / Raast / online / credit, with who-took-it and which-shift on every row. History backfilled so tender reporting is uniform.
- 03 **Money mutations move into database transactions** (create, add-round, settle, void, shift open/close, stock adjust): row-locked, idempotent, totals recomputed server-side. Client write access to money tables is revoked — the rule is *invariants in the database; orchestration in server actions; nothing money-shaped in client code*.
- 04 **Staff are app-level identities, not login accounts.** The two existing logins become device tiers (terminal, admin); people are rows with a server-verified PIN and per-action permissions. Two-second switching, full attribution, no re-auth.
- 05 **Multi-branch readiness is two rules, nothing more:** a one-row branches table now; every new table and unique index carries the branch from day one. No branch UI until branch two is real.
- 06 **Shifts stamp, never derive.** Each payment records its shift at tender time — a tab opened in shift one and settled in shift three is correctly shift-three cash. Close computes expected cash in the database and freezes the Z-report snapshot forever.
- 07 **Invoice numbers come from a per-day counter** — sequential, unique, always stored. The timestamp-based numbers retire.
- 08 **The audit log is written inside the same transactions** that perform each action — capturing the person, the device and the reason. Generic change-capture triggers are rejected: they can only ever see the shared device account, never the human.

How it should feel, and what comes first

Workflow & interface direction

- **The sidebar grows into module groups** as phases land — Service · Kitchen · Money · Menu · Inventory · Procurement · Insights · Admin — with terminals seeing only their tier.
- **Settings becomes a Master-Settings switchboard**, sectioned like Blink's: business · till behaviour · KOT & printing · shift & cash · tax & payments · alerts. Each phase adds its section.
- **At the till:** an active-cashier chip with a lock button (tap → PIN pad → switched), an active-shift indicator, a channel picker beside order type, charges and split tender inside the settle flow, and one reusable manager-approval pattern.
- **Keep** the black theme, the brand orange, tabs-and-rounds, the kitchen sounds. **Fix** contrast on orange, small touch targets, dialog keyboard behaviour.

Priority matrix

	NOW — P0	FOUNDATIONS — P1·P2	BUILD-OUT — P3·P4·P5	GROWTH — P6
CRITICAL	Settle guard · KDS query	Line items · payments · transactions · staff PINs · shifts	—	—
HIGH	Idempotency · discounts · revenue & timezone · PIN rule · logout · reprint · export	Invoice counters · audit log · permissions · staff screen	SQL reports · charges · presets · channels · customers · tables · stations · KOT · inventory core · food cost · procurement	QR ordering
MEDIUM	Discount reset · contrast	Waiters merge · expenses	Vouchers · split-by-item · riders · deals · credit · notes · live KDS · prep metrics · auto-86	Notifications · Foodpanda API
LOW	—	—	—	Loyalty · branch #2

Sequencing, commitment, horizon

The six rework traps (build order is the point)

- 01 Inventory, KOT routing, item-level splitting or SQL analytics built against the JSON blob is guaranteed rework — line items land first.
- 02 Patching settle and round-append in client code and then converting to transactions is the same function written twice — convert once, in P1.
- 03 Shifts before the payments ledger forces drawer maths onto a field that cannot express split tender.
- 04 Per-employee login accounts would be ripped out the first week the floor demands two-second switching — decide the session model first (done, §7.4).
- 05 Any new table or unique index without the branch dimension recreates the retrofit this plan exists to avoid.
- 06 Approval workflows before roles exist have no approver — sequence after P2.

Delivery risks

Live-database migrations (mitigated: rehearsal on a branch database, reconciliation counts, additive-only changes, versioned functions) · database-side logic is harder to unit test (small single-purpose functions, SQL test scripts) · KOT needs printer hardware per station (lead time) · Foodpanda depends on partner access (external, unbounded) · shifts and waste discipline are *operational* changes — staff training decides whether the data is real · single-developer bandwidth — every phase ships standalone value, so the roadmap survives interruption.

Recommended immediate commitment

Commit now to P0 + P1, then P2. P0 (a week): the twelve stabilisation fixes — the double-billing paths close, the numbers become honest. P1: the transactional core — little visible change, everything unblocked. P2: named cashiers, a float, X/Z reports, over-short — the moment the product visibly levels up, and the right point to reassess pace before P3–P5.

Long-term vision

Flames POS as a restaurant operating system for Pakistani single-brand operators: full online ordering with payment once the merchant account lands (Raast rails already half-built), Foodpanda orders landing automatically, a second branch activated by inserting a row, rider dispatch with live status, loyalty on the customer ledger, WhatsApp order-ready and day-end summaries, an offline-first till, and a BI layer reading the same spine of line items, payments and stock movements — Blink's A-to-Z lifecycle, owned outright: **menu → shift → order → KOT → payment → stock → purchase → report.**